Cognizant - UpGrad Training Program
WEEKLY CODING SUBMISSION
Name : Durgesh Vijay Bhangale
Week : Week 2
Trainer : Kunal

Day 1 : Bootstrap And JQuery(23-02-2026)

Practice Question 1 (Grid System + Responsive Layout)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Durgesh Product Section</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <div class="container mt-4">
    <h2 class="text-center mb-4">Our Products</h2>
    <div class="row">

      <div class="col-lg-3 col-md-6 col-12 mb-3">
        <div class="card h-100 text-center">
          <div class="card-body">
            <h5 class="card-title">Shirt</h5>
            <p class="card-text">Price: ₹500</p>
            <button class="btn btn-primary">Buy Now</button>
          </div>
        </div>
      </div>

      <div class="col-lg-3 col-md-6 col-12 mb-3">
        <div class="card h-100 text-center">
          <div class="card-body">
            <h5 class="card-title">T-Shirt</h5>
            <p class="card-text">Price: ₹750</p>
            <button class="btn btn-primary">Buy Now</button>
          </div>
        </div>
      </div>

      <div class="col-lg-3 col-md-6 col-12 mb-3">
        <div class="card h-100 text-center">
          <div class="card-body">
            <h5 class="card-title">Jeans</h5>
            <p class="card-text">Price: ₹1200</p>
            <button class="btn btn-primary">Buy Now</button>
```

```
        </div>
      </div>
    </div>

    <div class="col-lg-3 col-md-6 col-12 mb-3">
      <div class="card h-100 text-center">
        <div class="card-body">
          <h5 class="card-title">Sneakers</h5>
          <p class="card-text">Price: ₹1500</p>
          <button class="btn btn-primary">Buy Now</button>
        </div>
      </div>
    </div>
  </div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Practice Question 2 (Bootstrap Components)

Create a **Bootstrap Navbar**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Durgesh MyShop Navbar</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>

  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">

      <a class="navbar-brand" href="#">Durgesh's Shop</a>

      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>

      <div class="collapse navbar-collapse" id="navbarNav">
```

```
      <ul class="navbar-nav me-auto">
        <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Products</a></li>
        <li class="nav-item"><a class="nav-link" href="#">About</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
      </ul>

      <button class="btn btn-outline-light">Login</button>
     </div>
   </div>
  </nav>

  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Practice Question 3 (Utilities + Advanced Features)

Create a **"Customer Support" section** using Bootstrap

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Customer Support Section</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <div class="container my-4">
    <div class="bg-light text-center p-4 m-3 border rounded">
      <h2>Customer Support</h2>
      <p>Need help? Access our support menu for FAQs, contact options, and issue reporting.</p>
      <button class="btn btn-primary" type="button" data-bs-toggle="offcanvas" data-bs-
target="#supportMenu">
        Support Menu
      </button>
    </div>
  </div>
  <div class="offcanvas offcanvas-end" tabindex="-1" id="supportMenu">
    <div class="offcanvas-header">
      <h5 class="offcanvas-title">Support Menu</h5>
      <button type="button" class="btn-close" data-bs-dismiss="offcanvas"></button>
    </div>
    <div class="offcanvas-body">
      <ul class="list-group">
        <li class="list-group-item">FAQs</li>
```

```
      <li class="list-group-item">Contact Support</li>
      <li class="list-group-item">Report Issue</li>
     </ul>
    </div>
  </div>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Practice Question 4: Responsive Personal Portfolio Homepage

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Durgesh Portfolio</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
      <a class="navbar-brand" href="#">MyPortfolio</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Skills</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Projects</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
        </ul>
      </div>
    </div>
  </nav>
  <div class="container text-center my-5">
    <h1 class="mb-3">Durgesh Bhangale</h1>
    <p class="lead">Entry-Level Software Engineer | Python Developer</p>
    <button class="btn btn-primary">Download Resume</button>
  </div>

  <div class="container my-5">
    <h2 class="text-center mb-4">Skills</h2>
```

```
    <div class="row">
      <div class="col-lg-4 col-md-6 col-sm-12 mb-3">
        <div class="card h-100 text-center">
          <div class="card-body">
            <h5 class="card-title">Python</h5>
            <p class="card-text">Experienced in scripting, automation, and data analysis.</p>
          </div>
        </div>
      </div>
      <div class="col-lg-4 col-md-6 col-sm-12 mb-3">
        <div class="card h-100 text-center">
          <div class="card-body">
            <h5 class="card-title">SQL & Databases</h5>
            <p class="card-text">Strong knowledge of relational databases and queries.</p>
          </div>
        </div>
      </div>
      <div class="col-lg-4 col-md-6 col-sm-12 mb-3">
        <div class="card h-100 text-center">
          <div class="card-body">
            <h5 class="card-title">Cloud Fundamentals</h5>
            <p class="card-text">Basic understanding of Azure Cloud services.</p>
          </div>
        </div>
      </div>
    </div>
  </div>
  <div class="container my-5">
    <h2 class="text-center mb-4">Contact</h2>
    <div class="row justify-content-center">
      <div class="col-md-6 col-sm-12 text-center">
        <p>Email: <a href="mailto:durgesh@example.com">durgesh@gmail.com</a></p>
        <p>Phone: +91-9876543210</p>
        <button class="btn btn-success">Get in Touch</button>
      </div>
    </div>
  </div>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Practice Question 5: College Event Management Dashboard

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>College Event Management Dashboard</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">D y patil College Events</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item"><a class="nav-link active" href="#">Dashboard</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Events</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
        </ul>
      </div>
    </div>
  </nav>
  <div class="container my-4">
    <div class="row">
      <div class="col-lg-8 col-md-7 col-sm-12 mb-3">
        <h3 class="mb-3">Event List</h3>
        <ul class="list-group">
          <li class="list-group-item">Tech Fest 2026 - 15th March</li>
          <li class="list-group-item">Cultural Night - 20th March</li>
          <li class="list-group-item">Sports Meet - 25th March</li>
        </ul>
      </div>
      <div class="col-lg-4 col-md-5 col-sm-12 mb-3 text-center">
        <h3 class="mb-3">Manage Events</h3>
        <button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#addEventModal">
          Add New Event
        </button>
      </div>
    </div>
  </div>
  <div class="modal fade" id="addEventModal" tabindex="-1">
    <div class="modal-dialog">
      <div class="modal-content">
        <div class="modal-header">
          <h5 class="modal-title">Add Event</h5>
```

```
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <form>
          <div class="mb-3">
            <label for="eventName" class="form-label">Event Name</label>
            <input type="text" class="form-control" id="eventName" required>
            <div class="invalid-feedback">Please enter event name.</div>
          </div>
          <div class="mb-3">
            <label for="eventDate" class="form-label">Event Date</label>
            <input type="date" class="form-control" id="eventDate" required>
            <div class="invalid-feedback">Please select a date.</div>
          </div>
          <div class="mb-3">
            <label for="eventDesc" class="form-label">Description</label>
            <textarea class="form-control" id="eventDesc" rows="3"></textarea>
          </div>
          <button type="submit" class="btn btn-success">Save Event</button>
        </form>
      </div>
    </div>
  </div>
 </div>
 <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

JQUERY

Practice Question 1: Student Registration List

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Registration List</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body class="container my-4">
  <h2 class="mb-3">Student Registration</h2>
  <div class="mb-3">
    <input type="text" id="studentName" class="form-control mb-2" placeholder="Enter Student
Name">
    <input type="text" id="rollNumber" class="form-control mb-2" placeholder="Enter Roll Number">
    <button id="addStudent" class="btn btn-primary">Add Student</button>
  </div>
  <table class="table table-bordered mt-4">
    <thead class="table-dark">
      <tr>
        <th>Student Name</th>
        <th>Roll Number</th>
      </tr>
    </thead>
    <tbody id="studentList"></tbody>
  </table>
  <script>
    $(document).ready(function() {
      $("#addStudent").click(function() {
        let name = $("#studentName").val();
        let roll = $("#rollNumber").val();
        if(name && roll) {
          $("#studentList").append(
            `<tr><td>${name}</td><td>${roll}</td></tr>`
          );
          $("#studentName").val("");
          $("#rollNumber").val("");
        } else {
          alert("Please enter both Name and Roll Number!");
        }
      });
      $("#studentList").on("click", "tr", function() {
        $(this).remove();
      });
    });
  </script>
</body>
</html>
```

Practice Question 2: Live Character Counter

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Live Character Counter</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body class="container my-4">
  <h2 class="mb-3">Message Box</h2>
  <textarea id="messageBox" class="form-control mb-2" rows="4" placeholder="Type your
message..."></textarea>
  <p id="charCount" class="fw-bold">Characters: 0</p>
  <script>
    $(document).ready(function() {
      $("#messageBox").keyup(function() {
        let textLength = $(this).val().length;
        $("#charCount").text("Characters: " + textLength);
        if(textLength > 100) {
          $("#charCount").text("Limit exceeded! Characters: " + textLength);
          $("#charCount").css("color", "red");
        } else {
          $("#charCount").css("color", "black");
        }
      });
    });
  </script>
</body>
</html>
```

Practice Question 3: Simple Shopping Cart Items

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Simple Shopping Cart</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body class="container my-4">
  <h2 class="mb-3">Shopping Cart</h2>
  <select id="itemSelect" class="form-select mb-2">
    <option value="Apple">Apple</option>
    <option value="Mango">Mango</option>
    <option value="Banana">Banana</option>
  </select>
  <button id="addToCart" class="btn btn-primary mb-3">Add to Cart</button>
  <ul id="cartList" class="list-group mb-3"></ul>
```

```
  <p id="totalItems" class="fw-bold">Total Items: 0</p>

  <script>
    $(document).ready(function() {
      $("#addToCart").click(function() {
        let selectedItem = $("#itemSelect").val();

        $("#cartList").append(
          `<li class="list-group-item d-flex justify-content-between align-items-center">
            ${selectedItem}
            <button class="btn btn-danger btn-sm removeItem">Remove</button>
          </li>`
        );

        updateTotal();
      });
      $("#cartList").on("click", ".removeItem", function() {
        $(this).closest("li").remove();
        updateTotal();
      });
      function updateTotal() {
        let count = $("#cartList li").length;
        $("#totalItems").text("Total Items: " + count);
      }
    });
  </script>
</body>
</html>
```

DAY 2(24-02-2026)

QUESTION 1 (Database Design + Normalization + Constraints + Data Types)

A hospital wants to store patient appointment details.

```
CREATE TABLE Patients (
  PatientID INT PRIMARY KEY,
  PatientName VARCHAR(100) NOT NULL,
  PatientPhone VARCHAR(15) UNIQUE NOT NULL
);

CREATE TABLE Doctors (
  DoctorID INT PRIMARY KEY,
  DoctorName VARCHAR(100) NOT NULL,
  DoctorSpecialization VARCHAR(100) NOT NULL
);

CREATE TABLE Appointments (
  AppointmentID INT PRIMARY KEY,
  PatientID INT NOT NULL,
  DoctorID INT NOT NULL,
  AppointmentDate DATE NOT NULL,
  Fees INT NOT NULL,
  FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
  FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)
);

INSERT INTO Patients (PatientID, PatientName, PatientPhone)
VALUES
(1, 'Ravi Kumar', '9876543210'),
(2, 'Sneha Patil', '9123456789'),
(3, 'Amit Sharma', '9988776655'),
(4, 'Priya Desai', '9090909090'),
(5, 'Kunal Joshi', '9191919191');

INSERT INTO Doctors (DoctorID, DoctorName, DoctorSpecialization)
VALUES
(1, 'Dr. Mehta', 'Cardiology'),
(2, 'Dr. Rao', 'Orthopedics'),
(3, 'Dr. Kapoor', 'Dermatology'),
(4, 'Dr. Singh', 'Neurology'),
(5, 'Dr. Iyer', 'Pediatrics');

INSERT INTO Appointments (PatientID, DoctorID, AppointmentDate, Fees)
VALUES
(1, 1, '2026-02-20', 500),
(2, 2, '2026-02-21', 700),
(3, 3, '2026-02-22', 400),
```

```
(4, 4, '2026-02-23', 800),
(5, 5, '2026-02-24', 600);

SELECT
  p.PatientName,
  d.DoctorName,
  d.DoctorSpecialization,
  a.AppointmentDate,
  a.Fees
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID;
```

QUESTION 2 (Joins + Filtering + Sorting + Expressions)

```
CREATE TABLE Departments (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE Employees (
  EmpID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Salary INT NOT NULL,
  DeptID INT NOT NULL,
  FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

CREATE TABLE Projects (
  ProjectID INT PRIMARY KEY,
  ProjectName VARCHAR(100) NOT NULL,
  DeptID INT NOT NULL,
  FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

INSERT INTO Departments (DeptID, DeptName)
VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');

INSERT INTO Employees (EmpID, Name, Salary, DeptID)
VALUES
(101, 'Durgesh Bhangale', 60000, 1),
(102, 'Sneha Patil', 55000, 2),
(103, 'Amit Sharma', 45000, 1),
(104, 'Priya Desai', 70000, 3),
```

```
(105, 'Kunal Joshi', 52000, 2);

INSERT INTO Projects (ProjectID, ProjectName, DeptID)
VALUES
(201, 'Website Development', 1),
(202, 'Recruitment Portal', 2),
(203, 'Payroll System', 3),
(204, 'Mobile App', 1);

SELECT
  e.EmpID,
  e.Name,
  d.DeptName,
  p.ProjectName,
  (e.Salary * 12) AS AnnualSalary
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
JOIN Projects p ON d.DeptID = p.DeptID
WHERE e.Salary > 50000
ORDER BY e.Salary DESC;
```

QUESTION 3 (GROUP BY + HAVING + Subquery)

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerName VARCHAR(100) NOT NULL,
  ProductName VARCHAR(100) NOT NULL,
  Category VARCHAR(50) NOT NULL,
  Quantity INT NOT NULL,
  Price INT NOT NULL
);

INSERT INTO Orders (OrderID, CustomerName, ProductName, Category, Quantity, Price)
VALUES
(1, 'Durgesh Bhangale', 'Laptop', 'Electronics', 1, 50000),
(2, 'Sneha Patil', 'T-Shirt', 'Clothing', 2, 800),
(3, 'Amit Sharma', 'Rice Bag', 'Grocery', 5, 500),
(4, 'Priya Desai', 'Mobile', 'Electronics', 1, 30000),
(5, 'Kunal Joshi', 'Jeans', 'Clothing', 1, 1200),
(6, 'Ravi Kumar', 'Headphones', 'Electronics', 2, 2000),
(7, 'Sneha Patil', 'Milk Pack', 'Grocery', 10, 50),
(8, 'Amit Sharma', 'Shoes', 'Clothing', 1, 2500);

SELECT
  OrderID,
  CustomerName,
  ProductName,
```

```
  Category,
  Quantity,
  Price,
  (Quantity * Price) AS TotalAmount
FROM Orders;

SELECT Category, SUM(Quantity * Price) AS TotalSales
FROM Orders
GROUP BY Category
ORDER BY TotalSales DESC
LIMIT 1;

SELECT CustomerName, SUM(Quantity * Price) AS TotalSpent
FROM Orders
GROUP BY CustomerName
HAVING SUM(Quantity * Price) > (
  SELECT AVG(TotalCustomerSpent)
  FROM (
    SELECT SUM(Quantity * Price) AS TotalCustomerSpent
    FROM Orders
    GROUP BY CustomerName
  ) AS SubQuery
);

SELECT Category, SUM(Quantity * Price) AS TotalSales
FROM Orders
GROUP BY Category
HAVING SUM(Quantity * Price) > 5000;
```

QUESTION 4 (Set Operators + Update/Delete + Complex Logic)

```
CREATE TABLE BatchA (
  StudentName VARCHAR(100) PRIMARY KEY
);

CREATE TABLE BatchB (
  StudentName VARCHAR(100) PRIMARY KEY
);

INSERT INTO BatchA (StudentName)
VALUES
('Durgesh'),
('Sneha'),
('Amit'),
('Priya'),
('Kunal'),
('Megha');

INSERT INTO BatchB (StudentName)
```

```sql
VALUES
('Durgesh'),   -- common
('Kunal'),  -- common
('Arjun'),
('Neha'),
('Suresh'),
('Pooja');

SELECT StudentName FROM BatchA
INTERSECT
SELECT StudentName FROM BatchB;

SELECT StudentName FROM BatchA
UNION
SELECT StudentName FROM BatchB;
SELECT StudentName FROM BatchA
EXCEPT
SELECT StudentName FROM BatchB;

CREATE TABLE Fees (
  StudentName VARCHAR(100) PRIMARY KEY,
  FeesPaid INT NOT NULL,
  CourseName VARCHAR(50) NOT NULL
);

INSERT INTO Fees (StudentName, FeesPaid, CourseName)
VALUES
('Durgesh', 1500, 'SQL'),
('Sneha', 4000, 'Python'),
('Amit', 2500, 'SQL'),
('Priya', 3500, 'Java'),
('Kunal', 5000, 'SQL'),
('Megha', 1800, 'SQL');

UPDATE Fees
SET FeesPaid = FeesPaid + 1000
WHERE CourseName = 'SQL';

DELETE FROM Fees
WHERE FeesPaid < 2000;

SELECT
  StudentName,
  FeesPaid,
  CASE
    WHEN FeesPaid >= 3000 THEN 'Paid'
    ELSE 'Due'
  END AS Status
FROM Fees;
```

QUESTION 5 — E-Commerce Order Integrity (Normalization + Constraints + Subquery)

```
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(100) NOT NULL,
  CustomerEmail VARCHAR(100) NOT NULL
);

CREATE TABLE Categories (
  CategoryID INT PRIMARY KEY,
  CategoryName VARCHAR(50) NOT NULL
);

CREATE TABLE Products (
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(100) NOT NULL,
  CategoryID INT NOT NULL,
  Price INT NOT NULL,
  FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);

CREATE TABLE Orders1 (
  OrderID INT PRIMARY KEY,
  CustomerID INT NOT NULL,
  OrderDate DATE NOT NULL,
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE OrderDetails (
  OrderDetailID INT PRIMARY KEY,
  OrderID INT NOT NULL,
  ProductID INT NOT NULL,
  Quantity INT NOT NULL,
  FOREIGN KEY (OrderID) REFERENCES Orders1(OrderID),
  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

INSERT INTO Customers VALUES
(1, 'Durgesh Bhangale', 'Durgesh@gmail.com'),
(2, 'Sneha Patil', 'sneha@gmail.com'),
(3, 'Amit Sharma', 'amit@gmail.com'),
(4, 'Priya Desai', 'priya@gmail.com'),
(5, 'Kunal Joshi', 'kunal@gmail.com');

INSERT INTO Categories VALUES
(1, 'Electronics'),
(2, 'Clothing'),
(3, 'Grocery'),
(4, 'Books'),
(5, 'Sports');
```

```sql
INSERT INTO Products VALUES
(101, 'Laptop', 1, 50000),
(102, 'T-Shirt', 2, 800),
(103, 'Rice Bag', 3, 500),
(104, 'Novel', 4, 300),
(105, 'Football', 5, 1000);

INSERT INTO Orders1 VALUES
(201, 1, '2026-02-20'),
(202, 2, '2026-02-21'),
(203, 3, '2026-02-22'),
(204, 4, '2026-02-23'),
(205, 5, '2026-02-24'),
(206, 1, '2026-02-25'),
(207, 2, '2026-02-26'),
(208, 3, '2026-02-27');

INSERT INTO OrderDetails VALUES
(301, 201, 101, 1),
(302, 202, 102, 2),
(303, 203, 103, 5),
(304, 204, 104, 3),
(305, 205, 105, 2),
(306, 206, 102, 1),
(307, 207, 103, 2),
(308, 208, 104, 1);

SELECT
  c.CustomerName,
  p.ProductName,
  cat.CategoryName,
  od.Quantity,
  (od.Quantity * p.Price) AS TotalAmount
FROM OrderDetails od
JOIN Orders1 o ON od.OrderID = o.OrderID
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories cat ON p.CategoryID = cat.CategoryID;

SELECT c.CustomerName
FROM Orders1 o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories cat ON p.CategoryID = cat.CategoryID
JOIN Customers c ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerName
HAVING COUNT(DISTINCT cat.CategoryID) > 1;
```

```
SELECT p.ProductName, SUM(od.Quantity) AS TotalSold
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductName
ORDER BY TotalSold DESC
LIMIT 1;

DELETE FROM Products
WHERE ProductID NOT IN (
  SELECT DISTINCT ProductID FROM OrderDetails
);
```

QUESTION 6 — Banking System (Transactions + Complex Logic)

```
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Phone VARCHAR(15) UNIQUE NOT NULL
);

CREATE TABLE Accounts (
  AccountID INT PRIMARY KEY,
  CustomerID INT NOT NULL,
  Balance INT NOT NULL CHECK (Balance >= 0),
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE Transactions (
  TransactionID INT PRIMARY KEY,
  AccountID INT NOT NULL,
  Type VARCHAR(10) NOT NULL,
  Amount INT NOT NULL,
  Date DATE NOT NULL,
  FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)
);

INSERT INTO Customers VALUES
(1, 'Durgesh Bhangale', '9876543210'),
(2, 'Sneha Patil', '9123456789'),
(3, 'Amit Sharma', '9988776655'),
(4, 'Priya Desai', '9090909090'),
(5, 'Kunal Joshi', '9191919191');

INSERT INTO Accounts VALUES
(101, 1, 120000),
(102, 1, 50000),
(103, 2, 80000),
(104, 3, 30000),
```

```
(105, 4, 150000),
(106, 5, 0);

INSERT INTO Transactions VALUES
(201, 101, 'Credit', 20000, '2026-02-01'),
(202, 101, 'Debit', 5000, '2026-02-02'),
(203, 102, 'Credit', 10000, '2026-02-03'),
(204, 103, 'Debit', 8000, '2026-02-04'),
(205, 104, 'Credit', 15000, '2026-02-05'),
(206, 105, 'Debit', 20000, '2026-02-06'),
(207, 105, 'Credit', 50000, '2026-02-07'),
(208, 106, 'Credit', 1000, '2026-02-08'),
(209, 102, 'Debit', 2000, '2026-02-09'),
(210, 103, 'Credit', 12000, '2026-02-10');

SELECT
  c.Name AS CustomerName,
  a.AccountID,
  SUM(CASE WHEN t.Type = 'Credit' THEN t.Amount ELSE 0 END) AS TotalCredits,
  SUM(CASE WHEN t.Type = 'Debit' THEN t.Amount ELSE 0 END) AS TotalDebits,
  a.Balance AS CurrentBalance
FROM Accounts a
JOIN Customers c ON a.CustomerID = c.CustomerID
LEFT JOIN Transactions t ON a.AccountID = t.AccountID
GROUP BY c.Name, a.AccountID, a.Balance;

SELECT DISTINCT c.Name
FROM Transactions t
JOIN Accounts a ON t.AccountID = a.AccountID
JOIN Customers c ON a.CustomerID = c.CustomerID
WHERE t.Amount > (
  SELECT AVG(Amount) FROM Transactions
);

UPDATE Accounts
SET Balance = Balance + (Balance * 0.02)
WHERE Balance > 100000;

DELETE FROM Accounts
WHERE Balance = 0
AND NOT EXISTS (
  SELECT 1 FROM Transactions t
  WHERE t.AccountID = Accounts.AccountID
);
```

QUESTION 7 — University Grading System (CASE + HAVING + Ranking Logic)

```
CREATE TABLE Students1 (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  DeptID INT NOT NULL
);

CREATE TABLE Departments1 (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(100) NOT NULL
);

CREATE TABLE Marks (
  StudentID INT NOT NULL,
  Subject VARCHAR(100) NOT NULL,
  Marks INT NOT NULL,
  FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);

INSERT INTO Departments1 VALUES
(1, 'Computer Science'),
(2, 'Mechanical'),
(3, 'Electronics');

INSERT INTO Students1 VALUES
(101, 'Durgesh Bhangale', 1),
(102, 'Sneha Patil', 1),
(103, 'Amit Sharma', 2),
(104, 'Priya Desai', 2),
(105, 'Kunal Joshi', 3),
(106, 'Megha Rao', 3),
(107, 'Arjun Singh', 1),
(108, 'Neha Kapoor', 2);

INSERT INTO Marks VALUES
(101, 'DBMS', 95),
(101, 'OS', 85),
(102, 'DBMS', 70),
(102, 'OS', 80),
(103, 'Thermo', 60),
(103, 'Mechanics', 65),
(104, 'Thermo', 75),
(104, 'Mechanics', 70),
(105, 'Circuits', 90),
(105, 'Signals', 85),
(106, 'Circuits', 55),
(106, 'Signals', 60),
(107, 'DBMS', 88),
(107, 'OS', 92),
```

```
(108, 'Thermo', 72),
(108, 'Mechanics', 68);

SELECT
  s.StudentID,
  s.Name,
  SUM(m.Marks) AS TotalMarks,
  CASE
    WHEN SUM(m.Marks) >= 90 THEN 'A'
    WHEN SUM(m.Marks) >= 75 THEN 'B'
    WHEN SUM(m.Marks) >= 60 THEN 'C'
    ELSE 'Fail'
  END AS Grade
FROM Students1 s
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY s.StudentID, s.Name;

SELECT d.DeptName, AVG(m.Marks) AS AvgMarks
FROM Students1 s
JOIN Departments1 d ON s.DeptID = d.DeptID
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY d.DeptName
ORDER BY AvgMarks DESC
LIMIT 1;

SELECT s.Name, s.DeptID, SUM(m.Marks) AS TotalMarks
FROM Students1 s
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY s.StudentID, s.Name, s.DeptID
HAVING SUM(m.Marks) > (
  SELECT AVG(student_total) FROM (
    SELECT SUM(m2.Marks) AS student_total
    FROM Students1 s2
    JOIN Marks m2 ON s2.StudentID = m2.StudentID
    WHERE s2.DeptID = s.DeptID
    GROUP BY s2.StudentID
  )
);

SELECT d.DeptName, AVG(m.Marks) AS AvgMarks
FROM Students1 s
JOIN Departments d ON s.DeptID = d.DeptID
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY d.DeptName
HAVING AVG(m.Marks) > 70;
```

QUESTION 8 — Inventory + Supplier System (Advanced JOIN + Set Logic)

```
CREATE TABLE Suppliers (
  SupplierID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL
);

CREATE TABLE Products1 (
  ProductID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  SupplierID INT,
  StockQty INT NOT NULL,
  FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID)
);

CREATE TABLE Orders2 (
  OrderID INT PRIMARY KEY,
  ProductID INT NOT NULL,
  QuantityOrdered INT NOT NULL,
  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

INSERT INTO Suppliers VALUES
(1, 'ABC Traders'),
(2, 'Global Supplies'),
(3, 'Tech World'),
(4, 'NoProducts Supplier');

INSERT INTO Products1 VALUES
(101, 'Laptop', 1, 80),
(102, 'Mouse', 1, 150),
(103, 'Keyboard', 2, 50),
(104, 'Monitor', 2, 600),
(105, 'Printer', 3, 180);

INSERT INTO Orders2 VALUES
(201, 101, 100),
(202, 102, 50),
(203, 103, 70),
(204, 104, 200),
(205, 105, 190);

SELECT s.Name, COUNT(p.ProductID) AS ProductCount
FROM Suppliers s
JOIN Products1 p ON s.SupplierID = p.SupplierID
GROUP BY s.Name
HAVING COUNT(p.ProductID) > 1;

SELECT p.Name, p.StockQty, SUM(o.QuantityOrdered) AS TotalOrdered
FROM Products1 p
```

```
JOIN Orders2 o ON p.ProductID = o.ProductID
GROUP BY p.Name, p.StockQty
HAVING p.StockQty < SUM(o.QuantityOrdered);

SELECT s.Name
FROM Suppliers s
LEFT JOIN Products1 p ON s.SupplierID = p.SupplierID
WHERE p.ProductID IS NULL;

UPDATE Products1
SET StockQty = StockQty + 50
WHERE StockQty < 100;

DELETE FROM Suppliers s
WHERE NOT EXISTS (
  SELECT 1 FROM Products1 p
  WHERE p.SupplierID = s.SupplierID
);

SELECT
  p.Name AS ProductName,
  p.StockQty,
  CASE
    WHEN p.StockQty >= 500 THEN 'High Stock'
    WHEN p.StockQty BETWEEN 200 AND 499 THEN 'Medium Stock'
    ELSE 'Low Stock'
  END AS StockStatus
FROM Products1 p;
```

QUESTION 9 — Movie Streaming Platform (Tricky Analytics)

```
CREATE TABLE Users (
  UserID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  SubscriptionType VARCHAR(50) NOT NULL
);

CREATE TABLE Movies (
  MovieID INT PRIMARY KEY,
  Title VARCHAR(100) NOT NULL,
  Genre VARCHAR(50) NOT NULL
);

CREATE TABLE WatchHistory (
  UserID INT NOT NULL,
  MovieID INT NOT NULL,
  WatchTime INT NOT NULL,
  FOREIGN KEY (UserID) REFERENCES Users(UserID),
```

```
  FOREIGN KEY (MovieID) REFERENCES Movies(MovieID)
);

INSERT INTO Users VALUES
(1, 'Durgesh Bhangale', 'Basic'),
(2, 'Sneha Patil', 'Standard'),
(3, 'Amit Sharma', 'Premium'),
(4, 'Priya Desai', 'Basic'),
(5, 'Kunal Joshi', 'Standard');

INSERT INTO Movies VALUES
(101, 'Inception', 'Sci-Fi'),
(102, 'Titanic', 'Romance'),
(103, 'Avengers', 'Action'),
(104, 'Interstellar', 'Sci-Fi'),
(105, 'Joker', 'Drama');

INSERT INTO WatchHistory VALUES
(1, 101, 120),
(1, 102, 90),
(2, 103, 150),
(2, 104, 100),
(3, 101, 200),
(3, 105, 80),
(4, 102, 60),
(4, 103, 70),
(5, 104, 110),
(5, 105, 95);

SELECT m.Title, SUM(w.WatchTime) AS TotalWatchTime
FROM Movies m
JOIN WatchHistory w ON m.MovieID = w.MovieID
GROUP BY m.Title
ORDER BY TotalWatchTime DESC
LIMIT 1;

SELECT u.Name
FROM Users u
JOIN WatchHistory w ON u.UserID = w.UserID
JOIN Movies m ON w.MovieID = m.MovieID
GROUP BY u.Name
HAVING COUNT(DISTINCT m.Genre) > 2;

UPDATE Users
SET SubscriptionType = 'Premium'
WHERE UserID IN (
  SELECT UserID
  FROM WatchHistory
  GROUP BY UserID
```

```
  HAVING SUM(WatchTime) > (
    SELECT AVG(UserTotal)
    FROM (
      SELECT SUM(WatchTime) AS UserTotal
      FROM WatchHistory
      GROUP BY UserID
    ) AS SubQuery
  )
);

SELECT m.Title
FROM Movies m
WHERE NOT EXISTS (
  SELECT 1 FROM WatchHistory w
  WHERE w.MovieID = m.MovieID
);
```

QUESTION 10 — HR Promotion Logic (Very Interview-Oriented)

```
CREATE TABLE Employees2 (
  EmpID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Salary INT NOT NULL,
  DeptID INT NOT NULL
);

CREATE TABLE Performance (
  EmpID INT PRIMARY KEY,
  Rating INT NOT NULL,
  FOREIGN KEY (EmpID) REFERENCES Employees(EmpID)
);

CREATE TABLE Departments2 (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(100) NOT NULL
);

INSERT INTO Departments2 VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');

INSERT INTO Employees2 VALUES
(101, 'Durgesh Bhangale', 50000, 1),
(102, 'Sneha Patil', 60000, 2),
(103, 'Amit Sharma', 45000, 1),
(104, 'Priya Desai', 70000, 3),
(105, 'Kunal Joshi', 55000, 2);
```

```sql
INSERT INTO Performance VALUES
(101, 5),
(102, 4),
(103, 3),
(104, 2),
(105, 1);

UPDATE Employees2
SET Salary = Salary + (Salary * 0.10)
WHERE EmpID IN (
  SELECT EmpID FROM Performance WHERE Rating >= 4
);

SELECT
  e.Name,
  d.DeptName,
  e.Salary,
  p.Rating,
  CASE
    WHEN p.Rating = 5 THEN 'Excellent'
    WHEN p.Rating = 4 THEN 'Good'
    WHEN p.Rating = 3 THEN 'Average'
    ELSE 'Poor'
  END AS PerformanceLevel
FROM Employees2 e
JOIN Departments2 d ON e.DeptID = d.DeptID
JOIN Performance p ON e.EmpID = p.EmpID;

SELECT d.DeptName, AVG(e.Salary) AS AvgSalary
FROM Employees2 e
JOIN Departments2 d ON e.DeptID = d.DeptID
GROUP BY d.DeptName
ORDER BY AvgSalary DESC
LIMIT 1;

SELECT e.Name, e.Salary, d.DeptName
FROM Employees2 e
JOIN Departments2 d ON e.DeptID = d.DeptID
WHERE e.Salary > (
  SELECT AVG(e2.Salary)
  FROM Employees2 e2
  WHERE e2.DeptID = e.DeptID
);

DELETE FROM Employees2
WHERE EmpID IN (
  SELECT EmpID FROM Performance WHERE Rating < 2
);
```

QUESTION 11 — Fraud Detection Logic

```
CREATE TABLE Transactions1 (
  TransactionID INT PRIMARY KEY,
  UserID INT NOT NULL,
  Amount INT NOT NULL,
  Location VARCHAR(100) NOT NULL,
  Date DATE NOT NULL
);

INSERT INTO Transactions1 VALUES
(1, 101, 50000, 'Pune', '2026-02-20'),
(2, 101, 120000, 'Mumbai', '2026-02-20'),
(3, 101, 30000, 'Pune', '2026-02-20'),
(4, 101, 25000, 'Pune', '2026-02-20'),
(5, 102, 40000, 'Delhi', '2026-02-21'),
(6, 102, 45000, 'Delhi', '2026-02-21'),
(7, 103, 150000, 'Chennai', '2026-02-22'),
(8, 104, 20000, 'Pune', '2020-01-15'),
(9, 105, 60000, 'Delhi', '2026-02-23'),
(10, 105, 70000, 'Mumbai', '2026-02-23');

SELECT UserID, Date, COUNT(*) AS TransactionCount
FROM Transactions1
GROUP BY UserID, Date
HAVING COUNT(*) > 3;

SELECT UserID
FROM Transactions1
GROUP BY UserID
HAVING COUNT(DISTINCT Location) > 1;

SELECT
  TransactionID,
  UserID,
  Amount,
  Location,
  Date,
  CASE
    WHEN Amount > 100000 THEN 'High Risk'
    WHEN UserID IN (
      SELECT UserID
      FROM Transactions1
      GROUP BY UserID
      HAVING COUNT(DISTINCT Location) > 1
    ) THEN 'Location Risk'
    ELSE 'Normal'
  END AS RiskFlag
FROM Transactions1;
```

Day 3(25-02-2026)

PROJECT 1 — Smart Hospital Management & Billing Analytics System

```
CREATE TABLE Patients (
  PatientID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Phone VARCHAR(15) UNIQUE,
  Email VARCHAR(100) UNIQUE,
  DOB DATE
);

CREATE TABLE Doctors (
  DoctorID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  SpecializationID INT,
  Salary DECIMAL(12,2),
  FOREIGN KEY (SpecializationID) REFERENCES Specializations(SpecializationID)
);

CREATE TABLE Specializations (
  SpecializationID INT PRIMARY KEY,
  SpecName VARCHAR(100) NOT NULL
);

CREATE TABLE Appointments (
  AppointmentID INT PRIMARY KEY,
  PatientID INT NOT NULL,
  DoctorID INT NOT NULL,
  AppointmentDate DATE NOT NULL,
  FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
  FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)
);

CREATE TABLE Treatments (
  TreatmentID INT PRIMARY KEY,
  AppointmentID INT NOT NULL,
  TreatmentName VARCHAR(100),
  FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID)
);

CREATE TABLE Bills (
  BillID INT PRIMARY KEY,
  AppointmentID INT NOT NULL,
  BillAmount DECIMAL(12,2) CHECK (BillAmount > 0),
  InsuranceCoveredAmount DECIMAL(12,2),
  FinalPayableAmount DECIMAL(12,2),
  FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID)
);
```

```sql
CREATE TABLE InsuranceProviders (
  ProviderID INT PRIMARY KEY,
  Name VARCHAR(100),
  Contact VARCHAR(50)
);

SELECT
  p.Name AS PatientName,
  d.Name AS DoctorName,
  s.SpecName AS Specialization,
  t.TreatmentName,
  a.AppointmentDate,
  b.BillAmount,
  b.InsuranceCoveredAmount,
  b.FinalPayableAmount
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Specializations s ON d.SpecializationID = s.SpecializationID
JOIN Treatments t ON a.AppointmentID = t.AppointmentID;

SELECT d.Name, SUM(b.BillAmount) AS TotalRevenue
FROM Doctors d
JOIN Appointments a ON d.DoctorID = a.DoctorID
JOIN Bills b ON a.AppointmentID = b.AppointmentID
GROUP BY d.Name
ORDER BY TotalRevenue DESC
LIMIT 3;

SELECT p.Name,
      CASE
        WHEN b.FinalPayableAmount > 0 THEN 'Pending'
        ELSE 'Paid'
      END AS PaymentStatus
FROM Patients p
JOIN Appointments a ON p.PatientID = a.PatientID
JOIN Bills b ON a.AppointmentID = b.AppointmentID;

SELECT s.SpecName, SUM(b.BillAmount) AS SpecRevenue
FROM Specializations s
JOIN Doctors d ON s.SpecializationID = d.SpecializationID
JOIN Appointments a ON d.DoctorID = a.DoctorID
JOIN Bills b ON a.AppointmentID = b.AppointmentID
GROUP BY s.SpecName
HAVING SUM(b.BillAmount) > (
  SELECT AVG(TotalRevenue)
  FROM (
    SELECT SUM(b2.BillAmount) AS TotalRevenue
```

```
    FROM Doctors d2
    JOIN Appointments a2 ON d2.DoctorID = a2.DoctorID
    JOIN Bills b2 ON a2.AppointmentID = b2.AppointmentID
    GROUP BY d2.SpecializationID
  ) AS SubQuery
);

DELETE FROM Patients p
WHERE NOT EXISTS (
  SELECT 1
  FROM Appointments a
  WHERE a.PatientID = p.PatientID
    AND a.AppointmentDate >= DATEADD(YEAR, -3, GETDATE())
);

UPDATE Doctors
SET Salary = Salary * 1.05
WHERE DoctorID IN (
  SELECT d.DoctorID
  FROM Doctors d
  JOIN Appointments a ON d.DoctorID = a.DoctorID
  JOIN Bills b ON a.AppointmentID = b.AppointmentID
  GROUP BY d.DoctorID
  HAVING SUM(b.BillAmount) > 1000000
);
```

PROJECT 2 — Online Marketplace with Seller Performance & Fraud Detection

```
CREATE TABLE Users (
  UserID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Email VARCHAR(100) UNIQUE NOT NULL,
  UserType VARCHAR(10) NOT NULL CHECK (UserType IN ('Buyer','Seller')),
  Phone VARCHAR(15)
);

CREATE TABLE Categories (
  CategoryID INT PRIMARY KEY,
  CategoryName VARCHAR(100) NOT NULL
);

CREATE TABLE Products (
  ProductID INT PRIMARY KEY,
  SellerID INT NOT NULL,
  CategoryID INT NOT NULL,
  ProductName VARCHAR(100) NOT NULL,
  Price DECIMAL(10,2) CHECK (Price > 0),
  FOREIGN KEY (SellerID) REFERENCES Users(UserID),
  FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);
```

```sql
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  BuyerID INT NOT NULL,
  SellerID INT NOT NULL,
  OrderDate DATE NOT NULL,
  RefundRequested BOOLEAN DEFAULT 0,
  FOREIGN KEY (BuyerID) REFERENCES Users(UserID),
  FOREIGN KEY (SellerID) REFERENCES Users(UserID)
);

CREATE TABLE OrderDetails (
  OrderDetailID INT PRIMARY KEY,
  OrderID INT NOT NULL,
  ProductID INT NOT NULL,
  Quantity INT CHECK (Quantity > 0),
  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE,
  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

CREATE TABLE Payments (
  PaymentID INT PRIMARY KEY,
  OrderID INT NOT NULL,
  Amount DECIMAL(10,2) CHECK (Amount > 0),
  PaymentDate DATE NOT NULL,
  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);

CREATE TABLE Reviews (
  ReviewID INT PRIMARY KEY,
  ProductID INT NOT NULL,
  BuyerID INT NOT NULL,
  Rating INT CHECK (Rating BETWEEN 1 AND 5),
  Comment TEXT,
  FOREIGN KEY (ProductID) REFERENCES Products(ProductID),
  FOREIGN KEY (BuyerID) REFERENCES Users(UserID)
);

SELECT u.Name AS SellerName,
      COUNT(DISTINCT p.ProductID) AS TotalProducts,
      COUNT(DISTINCT o.OrderID) AS TotalOrders,
      SUM(pay.Amount) AS TotalRevenue,
      AVG(r.Rating) AS AverageRating,
      CASE
        WHEN SUM(pay.Amount) > 500000 THEN 'Platinum'
        WHEN SUM(pay.Amount) > 200000 THEN 'Gold'
        ELSE 'Silver'
      END AS PerformanceLevel
FROM Users u
JOIN Products p ON u.UserID = p.SellerID
JOIN OrderDetails od ON p.ProductID = od.ProductID
JOIN Orders o ON od.OrderID = o.OrderID
JOIN Payments pay ON o.OrderID = pay.OrderID
LEFT JOIN Reviews r ON p.ProductID = r.ProductID
```

```
WHERE u.UserType = 'Seller'
GROUP BY u.Name;

SELECT u.Name AS BuyerName
FROM Users u
JOIN Orders o ON u.UserID = o.BuyerID
GROUP BY u.UserID, u.Name, DATE(o.OrderDate)
HAVING COUNT(o.OrderID) > 5
  OR (
    SELECT COUNT(*) FROM Orders o2 WHERE o2.BuyerID = u.UserID AND o2.RefundRequested = 1
   ) > (
    SELECT COUNT(*)/2 FROM Orders o3 WHERE o3.BuyerID = u.UserID
   );

SELECT c.CategoryName, SUM(pay.Amount) AS TotalRevenue
FROM Categories c
JOIN Products p ON c.CategoryID = p.CategoryID
JOIN OrderDetails o ON p.ProductID = o.ProductID
JOIN Payments pay ON o.OrderID = pay.OrderID
GROUP BY c.CategoryName
ORDER BY TotalRevenue DESC
LIMIT 1;

DELETE FROM Users u
WHERE u.UserType = 'Seller'
 AND (NOT EXISTS (SELECT 1 FROM Products p WHERE p.SellerID = u.UserID)
    OR NOT EXISTS (SELECT 1 FROM Orders o WHERE o.SellerID = u.UserID
              AND o.OrderDate >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)));

UPDATE Products p
SET Price = Price * 0.9
WHERE ProductID IN (
 SELECT p2.ProductID
 FROM Products p2
 JOIN OrderDetails od ON p2.ProductID = od.ProductID
 GROUP BY p2.ProductID, p2.CategoryID
 HAVING SUM(od.Quantity) < (
  SELECT AVG(SUM(od2.Quantity))
  FROM Products p3
  JOIN OrderDetails od2 ON p3.ProductID = od2.ProductID
  WHERE p3.CategoryID = p2.CategoryID
  GROUP BY p3.ProductID
 )
);

SELECT p.ProductName
FROM Products p
WHERE NOT EXISTS (
 SELECT 1 FROM Reviews r WHERE r.ProductID = p.ProductID
);
```

PROJECT 3— "Global Ride-Sharing Analytics & Compliance System

```
CREATE TABLE Cities (
  CityID INT PRIMARY KEY,
  CityName VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Drivers (
  DriverID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Phone VARCHAR(15) UNIQUE,
  CityID INT NOT NULL,
  JoinDate DATE,
  FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);

CREATE TABLE Riders (
  RiderID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Phone VARCHAR(15) UNIQUE
);

CREATE TABLE Vehicles (
  VehicleID INT PRIMARY KEY,
  DriverID INT NOT NULL,
  Type VARCHAR(50),
  PlateNumber VARCHAR(20) UNIQUE,
  FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID)
);

CREATE TABLE Trips (
  TripID INT PRIMARY KEY,
  DriverID INT NOT NULL,
  RiderID INT NOT NULL,
  CityID INT NOT NULL,
  TripDate DATE NOT NULL,
  DistanceKM DECIMAL(8,2) CHECK (DistanceKM > 0),
  FareAmount DECIMAL(10,2) CHECK (FareAmount > 0),
  FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID),
  FOREIGN KEY (RiderID) REFERENCES Riders(RiderID),
  FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);

CREATE TABLE Payments1 (
  PaymentID INT PRIMARY KEY,
  TripID INT NOT NULL,
  PaymentMethod VARCHAR(20),
  PaidAmount DECIMAL(10,2) CHECK (PaidAmount > 0),
  FOREIGN KEY (TripID) REFERENCES Trips(TripID) ON DELETE CASCADE
);

CREATE TABLE DriverRatings (
  RatingID INT PRIMARY KEY,
  TripID INT NOT NULL,
  RatingScore INT CHECK (RatingScore BETWEEN 1 AND 5),
```

```
  FOREIGN KEY (TripID) REFERENCES Trips(TripID) ON DELETE CASCADE
);


SELECT d.Name AS DriverName, c.CityName,
      AVG(r.RatingScore) AS AvgRating,
      COUNT(t.TripID) AS TotalTrips,
      CASE
        WHEN AVG(r.RatingScore) < (
            SELECT AVG(r2.RatingScore)
            FROM Trips t2
            JOIN DriverRatings r2 ON t2.TripID = r2.TripID
            WHERE t2.CityID = d.CityID
          )
         AND COUNT(t.TripID) < (
            SELECT AVG(TripCount)
            FROM (
              SELECT COUNT(t3.TripID) AS TripCount
              FROM Trips t3
              WHERE t3.CityID = d.CityID
              GROUP BY t3.DriverID
            ) AS Sub
          )
        THEN 'Underperforming'
        WHEN AVG(r.RatingScore) >= 4.5 THEN 'Excellent'
        ELSE 'Stable'
      END AS Status
FROM Drivers d
JOIN Cities c ON d.CityID = c.CityID
JOIN Trips t ON d.DriverID = t.DriverID
JOIN DriverRatings r ON t.TripID = r.TripID
GROUP BY d.Name, c.CityName, d.CityID;

SELECT d.Name, COUNT(t.TripID) AS TripCount,
      MAX(t.FareAmount) AS MaxFare,
      AVG(t.FareAmount) AS AvgFare
FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
GROUP BY d.Name
HAVING COUNT(t.TripID) >= 5
   AND MAX(t.FareAmount) > 3 * AVG(t.FareAmount);

SELECT c.CityName
FROM Cities c
WHERE (
  SELECT SUM(FareAmount)
  FROM Trips t
  WHERE t.CityID = c.CityID
    AND t.DriverID IN (
      SELECT d.DriverID
      FROM Drivers d
      JOIN Trips t2 ON d.DriverID = t2.DriverID
      WHERE d.CityID = c.CityID
      GROUP BY d.DriverID
```

```
      HAVING SUM(t2.FareAmount) >= (
        SELECT 0.2 * SUM(FareAmount)
        FROM Trips t3
        WHERE t3.CityID = c.CityID
      )
    )
) > 0.6 * (
  SELECT SUM(FareAmount)
  FROM Trips t4
  WHERE t4.CityID = c.CityID
);

SELECT r.Name, t.TripDate
FROM Riders r
JOIN Trips t ON r.RiderID = t.RiderID
GROUP BY r.RiderID, r.Name, t.TripDate
HAVING COUNT(DISTINCT t.CityID) > 1;

UPDATE Trips
SET FareAmount = FareAmount * 1.05
WHERE DriverID IN (
  SELECT d.DriverID
  FROM Drivers d
  WHERE (
    SELECT COUNT(t.TripID)
    FROM Trips t
    WHERE t.DriverID = d.DriverID
  ) > (
    SELECT AVG(TripCount)
    FROM (
      SELECT COUNT(t2.TripID) AS TripCount
      FROM Trips t2
      WHERE t2.CityID = d.CityID
      GROUP BY t2.DriverID
    ) AS Sub
  )
  AND (
    SELECT AVG(r.RatingScore)
    FROM Trips t3
    JOIN DriverRatings r ON t3.TripID = r.TripID
    WHERE t3.DriverID = d.DriverID
  ) > (
    SELECT AVG(RatingScore) FROM DriverRatings
  )
);

DELETE FROM Drivers d
WHERE NOT EXISTS (SELECT 1 FROM Trips t WHERE t.DriverID = d.DriverID)
   OR NOT EXISTS (
       SELECT 1 FROM Trips t2
       WHERE t2.DriverID = d.DriverID
         AND t2.TripDate >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR)
   );
```

```sql
SELECT t.TripID, d.Name AS DriverName, c.CityName,
      CASE
        WHEN t.DistanceKM < 1
            AND t.FareAmount > (
              SELECT AVG(FareAmount) FROM Trips WHERE CityID = t.CityID
            ) THEN 'Suspicious'
        WHEN p.PaymentMethod = 'Cash' AND t.FareAmount > 5000 THEN 'High Risk'
        ELSE 'Normal'
      END AS RiskLevel
FROM Trips t
JOIN Drivers d ON t.DriverID = d.DriverID
JOIN Cities c ON t.CityID = c.CityID
JOIN Payments1 p ON t.TripID = p.TripID;

-- A = Drivers with rating ≥ 4
SELECT d.DriverID FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
JOIN DriverRatings r ON t.TripID = r.TripID
GROUP BY d.DriverID
HAVING AVG(r.RatingScore) >= 4

INTERSECT

-- B = Drivers with revenue above city average
SELECT d.DriverID FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
GROUP BY d.DriverID, d.CityID
HAVING SUM(t.FareAmount) > (
  SELECT AVG(TotalRevenue)
  FROM (
    SELECT SUM(t2.FareAmount) AS TotalRevenue
    FROM Trips t2
    WHERE t2.CityID = d.CityID
    GROUP BY t2.DriverID
  ) AS Sub
);

-- High revenue but low rating
SELECT d.DriverID FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
GROUP BY d.DriverID, d.CityID
HAVING SUM(t.FareAmount) > (
  SELECT AVG(TotalRevenue)
  FROM (
    SELECT SUM(t2.FareAmount) AS TotalRevenue
    FROM Trips t2
    WHERE t2.CityID = d.CityID
    GROUP BY t2.DriverID
  ) AS Sub
)
EXCEPT
SELECT d.DriverID FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
JOIN DriverRatings r ON t.TripID = r.TripID
```

```
GROUP BY d.DriverID
HAVING AVG(r.RatingScore) >= 4;

-- Union of strong drivers
SELECT d.DriverID FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
JOIN DriverRatings r ON t.TripID = r.TripID
GROUP BY d.DriverID
HAVING AVG
```

Day 4, 5 (26-02-2026 to 27-02-2026)

Project 1 — "Smart College Event Registration Portal"

1.Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Smart College Event Portal</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
<header>
  <h1>Welcome to Smart College Event Portal</h1>
  <nav>
    <a href="index.html">Home</a> |
    <a href="events.html">Events</a> |
    <a href="register.html">Register</a>
  </nav>
</header>

<section>
  <h2>About the Portal</h2>
  <p>This portal helps students view upcoming events, register easily, and track their progress.</p>
</section>

<section>
  <h2>Quick Links</h2>
  <ul>
    <li><a href="events.html">View Event Schedule</a></li>
    <li><a href="register.html">Register for Events</a></li>
  </ul>
</section>

<footer>
  <p>&copy; 2026 Smart College</p>
</footer>
```

```
</body>
</html>
```

2.event.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>College Events</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<header>
  <h1>Smart College Event Portal</h1>
  <nav>
    <a href="index.html">Home</a> >
    <a href="events.html">Events</a>
  </nav>
</header>

<section>
  <h2>Upcoming Events</h2>
  <ol>
    <li>Tech Fest</li>
    <li>Cultural Night</li>
    <li>Sports Meet</li>
  </ol>

  <article>
    <h3>Event Details</h3>
    <table border="1">
      <tr><th>Date</th><th>Venue</th><th>Seats Left</th></tr>
      <tr><td>2026-03-01</td><td>Auditorium</td><td><meter value="50" min="0"
max="100"></meter></td></tr>
      <tr><td>2026-03-05</td><td>Ground</td><td><meter value="20" min="0"
max="100"></meter></td></tr>
    </table>
  </article>

  <h3>Rules</h3>
  <ul>
    <li>Carry ID card</li>
    <li>One registration per student</li>
    <li>Follow event timings</li>
  </ul>
</section>

<footer>
  <p>&copy; 2026 Smart College</p>
```

```
</footer>
</body>
</html>
```

3.style.css

```
body { font-family: Arial; margin: 0; padding: 0; }
header, footer { background: #333; color: white; padding: 10px; }
nav a { margin-right: 10px; color: yellow; }
section { padding: 20px; }

table { width: 100%; border-collapse: collapse; }
td, th { padding: 8px; border: 1px solid #ccc; }

.flex-container { display: flex; justify-content: space-around; }

@media (max-width: 600px) {
  .flex-container { flex-direction: column; }
}
```

Script.js

```
let registeredStudents = [];

function register() {
  let name = document.getElementById("name").value;
  let email = document.getElementById("email").value;

  if (name === "" || email === "") {
    alert("Please fill all required fields!");
    return;
  }

  localStorage.setItem("studentName", name);

  document.body.innerHTML += `<p>Thank you ${name}, registration successful!</p>`;
}

document.getElementById("regForm").onchange = function() {
  let filled = 0;
  let inputs = document.querySelectorAll("#regForm input");
  inputs.forEach(i => { if (i.value !== "") filled++; });
  document.getElementById("formProgress").value = (filled / inputs.length) * 100;
};

function getLocation() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(showPosition, showError);
  } else {
    alert("Geolocation not supported");
```

```
  }
}

function showPosition(position) {
  document.body.innerHTML += `<p>Latitude: ${position.coords.latitude}, Longitude:
${position.coords.longitude}</p>`;
}

function showError() {
  alert("Permission denied or error fetching location");
}
```

Project 2 — “Personal Finance Tracker (Client-Side Only)”

1.index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Personal Finance Tracker</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
<header>
  <h1>Finance Tracker</h1>
</header>

<section class="flex-container">
  <article>
    <h2>Add Transaction</h2>
    <form id="transactionForm">
      <label>Date:</label>
      <input type="date" id="date" required><br>

      <label>Description:</label>
      <input type="text" id="desc" required><br>

      <label>Amount:</label>
      <input type="number" id="amount" required><br>

      <label>Category:</label>
      <input list="categories" id="category" required>
      <datalist id="categories">
        <option value="Food">
        <option value="Transport">
        <option value="Entertainment">
        <option value="Salary">
      </datalist><br>

      <button type="button" onclick="addTransaction()">Add</button>
```

```
    </form>
  </article>

  <article>
    <h2>Transactions</h2>
    <table id="transactionTable" border="1">
      <tr><th>Date</th><th>Description</th><th>Category</th><th>Amount</th></tr>
    </table>
    <p>Total Balance: <span id="balance">0</span></p>
    <progress id="savingsProgress" value="0" max="100"></progress><br>
    <meter id="expenseRatio" value="0" min="0" max="1"></meter>
  </article>
</section>

<section>
  <h2>Financial Tips</h2>
  <ol>
    <li>Save at least 20% of your income.</li>
    <li>Track expenses daily.</li>
    <li>Avoid unnecessary debt.</li>
  </ol>
</section>

<footer>
  <p>&copy; 2026 Finance Tracker</p>
</footer>
</body>
</html>
```

2.style.css

```
body { font-family: Arial; margin: 0; padding: 0; }
header { background: #333; color: white; padding: 10px; text-align: center; }
.flex-container { display: flex; justify-content: space-around; margin: 20px; }
article { padding: 20px; border: 1px solid #ccc; background: #f9f9f9; }
button:hover { background: green; color: white; }

@media (max-width: 600px) {
  .flex-container { flex-direction: column; }
}
```

3.script.js

```
let transactions = JSON.parse(localStorage.getItem("transactions")) || [];
function addTransaction() {
  let date = document.getElementById("date").value;
  let desc = document.getElementById("desc").value;
  let amount = parseFloat(document.getElementById("amount").value);
  let category = document.getElementById("category").value;
  if (!date || !desc || !amount || !category) {
```

```
    alert("Please fill all fields!");
    return;
  }
  let type = amount > 0 ? "Income" : "Expense";
  transactions.push({ date, desc, amount, category, type });
  localStorage.setItem("transactions", JSON.stringify(transactions));
  updateTable();
  calculateTotal();
}

function updateTable() {
  let table = document.getElementById("transactionTable");
  table.innerHTML =
"<tr><th>Date</th><th>Description</th><th>Category</th><th>Amount</th></tr>";
  transactions.forEach(t => {
    let row = table.insertRow();
    row.insertCell(0).innerText = t.date;
    row.insertCell(1).innerText = t.desc;
    row.insertCell(2).innerText = t.category;
    row.insertCell(3).innerText = t.amount;

    switch(t.category) {
      case "Food": row.style.background = "#ffe0e0"; break;
      case "Transport": row.style.background = "#e0f7ff"; break;
      case "Entertainment": row.style.background = "#f0e0ff"; break;
      case "Salary": row.style.background = "#e0ffe0"; break;
    }
  });
}
function calculateTotal() {
  let total = transactions.map(t => t.amount).reduce((a,b) => a+b, 0);
  document.getElementById("balance").innerText = total;
  document.getElementById("savingsProgress").value = Math.min(total, 1000);
  let income = transactions.filter(t => t.amount > 0).map(t => t.amount).reduce((a,b)=>a+b,0);
  let expense = transactions.filter(t => t.amount < 0).map(t =>
Math.abs(t.amount)).reduce((a,b)=>a+b,0);
  let ratio = income > 0 ? expense/income : 0;
  document.getElementById("expenseRatio").value = ratio;
}

// Load data on refresh
window.onload = function() {
  updateTable();
  calculateTotal();
};
```

Project 3 — “Multi-Page E-Learning Platform”

1.index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>E-Learning Dashboard</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
<header>
  <h1>Smart E-Learning Platform</h1>
  <nav>
    <a href="index.html">Home</a> |
    <a href="courses.html">Courses</a> |
    <a href="quiz.html">Quiz</a> |
    <a href="profile.html">Profile</a>
  </nav>
</header>
<section class="grid-container">
  <div class="card">
    <h2>Welcome Back!</h2>
    <p>Track your learning journey below.</p>
  </div>
  <div class="card">
    <h2>Overall Progress</h2>
    <progress id="courseProgress" value="40" max="100"></progress>
  </div>
</section>
<footer>
  <p>&copy; 2026 Smart College</p>
</footer>
</body>
</html>
```

2.profile.html

```
<section>
```

```
  <h2>User Profile</h2>
  <p>Name: <span id="userName">Durgesh</span></p>
  <p>Completed Courses:</p>
  <ul id="completedCourses"></ul>
  <p>Last Quiz Score: <span id="quizScore"></span></p>
  <p>Overall Progress:</p>
  <progress id="courseProgress" value="0" max="100"></progress>
</section>
<script src="script.js"></script>
```

3.courses.html

```
<section class="flex-container">
  <div class="course-card">
    <h3>HTML Basics</h3>
    <ol>
      <li>Introduction</li>
      <li>Tags</li>
      <li>Forms</li>
    </ol>
    <progress value="30" max="100"></progress><br>
    <button onclick="completeCourse('HTML Basics')">Mark Complete</button>
  </div>

  <div class="course-card">
    <h3>JavaScript</h3>
    <ol>
      <li>Variables</li>
      <li>Functions</li>
      <li>DOM</li>
    </ol>
    <progress value="60" max="100"></progress><br>
    <button onclick="completeCourse('JavaScript')">Mark Complete</button>
  </div>
</section>
```

4.quiz.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Quiz</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
<header>
  <h1>Quiz Page</h1>
  <nav>
    <a href="index.html">Home</a> |
```

```
    <a href="courses.html">Courses</a> |
    <a href="quiz.html">Quiz</a> |
    <a href="profile.html">Profile</a>
  </nav>
</header>

<section>
  <h2>Quiz</h2>
  <form id="quizForm">
    <div id="quizContainer"></div>
    <button type="submit">Submit Quiz</button>
  </form>
  <div id="result"></div>
</section>
</body>
</html>
```

4.style.css

```
body { font-family: Arial; margin: 0; padding: 0; background: #f4f4f9; }
header { background: #333; color: white; padding: 15px; text-align: center; }
nav a { margin: 0 10px; color: yellow; text-decoration: none; }
nav a:hover { color: orange; }

.grid-container { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; padding: 20px; }
.flex-container { display: flex; gap: 20px; padding: 20px; }
.course-card { background: white; padding: 15px; border-radius: 8px; box-shadow: 0 2px 5px
rgba(0,0,0,0.1); transition: transform 0.2s; }
.course-card:hover { transform: scale(1.05); }

@media (max-width: 600px) {
  .grid-container { grid-template-columns: 1fr; }
  .flex-container { flex-direction: column; }
}
```

5.script.js

```
let quizQuestions = [
  { q: "HTML stands for?", options: ["Hyper Text Markup Language", "Home Tool Markup Language"],
answer: 0 },
  { q: "Which keyword declares a variable in JS?", options: ["var", "int"], answer: 0 },
  { q: "CSS is used for?", options: ["Styling web pages", "Database management"], answer: 0 }
];
async function loadQuiz() {
  return new Promise(resolve => {
    setTimeout(() => resolve(quizQuestions), 500);
  });
}
async function showQuiz() {
  let questions = await loadQuiz();
```

```
  let container = document.getElementById("quizContainer");
  container.innerHTML = "";
  questions.forEach((q, i) => {
    let div = document.createElement("div");
    div.innerHTML = `<p>${q.q}</p>`;
    q.options.forEach((opt, j) => {
      div.innerHTML += `<label><input type="radio" name="q${i}" value="${j}"> ${opt}</label><br>`;
    });
    container.appendChild(div);
  });
}
function submitQuiz(event) {
  event.preventDefault();
  let score = 0;
  quizQuestions.forEach((q, i) => {
    let selected = document.querySelector(`input[name="q${i}"]:checked`);
    if (selected && parseInt(selected.value) === q.answer) score++;
  });

  let percentage = (score / quizQuestions.length) * 100;
  let grade = percentage >= 80 ? "A" : percentage >= 60 ? "B" : "C";
  let feedback = grade === "A" ? "Excellent!" : grade === "B" ? "Good job!" : "Keep practicing!";

  document.getElementById("result").innerHTML =
    `Score: ${percentage}% - Grade: ${grade} (${feedback})`;

  localStorage.setItem("quizScore", percentage);
  localStorage.setItem("quizCompleted", "true");
}
function completeCourse(courseName) {
  let completed = JSON.parse(localStorage.getItem("completedCourses")) || [];
  if (!completed.includes(courseName)) {
    completed.push(courseName);
    localStorage.setItem("completedCourses", JSON.stringify(completed));
  }
}

// Load Profile Data
function loadProfile() {
  let score = localStorage.getItem("quizScore");
  document.getElementById("quizScore").innerText = score ? score + "%" : "No quiz taken yet";
  let completed = JSON.parse(localStorage.getItem("completedCourses")) || [];
  let list = document.getElementById("completedCourses");
  list.innerHTML = "";
  completed.forEach(c => {
    let li = document.createElement("li");
    li.innerText = c;
    list.appendChild(li);
  });
```

```
  let progress = (completed.length / 2) * 100;
  document.getElementById("courseProgress").value = progress;
}
window.onload = function() {
  if (document.getElementById("quizContainer")) {
    showQuiz();
    document.getElementById("quizForm").addEventListener("submit", submitQuiz);
  }
  if (document.getElementById("quizScore")) {
    loadProfile();
  }
};
```

5.tests.test.js

```
function calculateGrade(score, total) {
  let percentage = (score / total) * 100;
  if(percentage >= 80) return "A";
  else if(percentage >= 60) return "B";
  else if(percentage >= 40) return "C";
  return "Fail";
}

test("Grade calculation", () => {
  expect(calculateGrade(8,10)).toBe("A");
});

test("Score percentage", () => {
  expect((7/10)*100).toBe(70);
});

test("Pass/fail logic", () => {
  expect(calculateGrade(2,10)).toBe("Fail");
});
```

Project 4 — “Geo-Based Task Manager with Async Storage Simulation”

1.index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Geo-Based Task Manager</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
```

```html
<header>
  <h1>Field Task Manager</h1>
</header>

<section class="grid-container">
  <article>
    <h2>Add Task</h2>
    <form id="taskForm">
      <label>Title:</label>
      <input type="text" id="title" required><br>

      <label>Due Date:</label>
      <input type="date" id="date" required><br>

      <label>Priority:</label>
      <input list="priorityList" id="priority" required>
      <datalist id="priorityList">
        <option value="High">
        <option value="Medium">
        <option value="Low">
      </datalist><br>

      <label>Completed:</label>
      <input type="checkbox" id="completed"><br>

      <label>Estimated Hours:</label>
      <input type="number" id="hours" min="1" max="24"><br>

      <button type="submit">Add Task</button>
    </form>
  </article>

  <article>
    <h2>Tasks</h2>
    <table id="taskTable" border="1">
      <tr><th>ID</th><th>Title</th><th>Priority</th><th>Location</th><th>Status</th></tr>
    </table>
    <p>Completion: <progress id="taskProgress" value="0" max="100"></progress></p>
    <p>Priority Load: <meter id="priorityMeter" value="0" min="0" max="1"></meter></p>
  </article>
</section>

<footer>
  <p>&copy; 2026 Geo Task Manager</p>
</footer>
</body>
</html>
```

2.style.css

```
body { font-family: Arial; margin: 0; background: #f4f4f9; }
header { background: #333; color: white; padding: 15px; text-align: center; }
.grid-container { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; padding: 20px; }
article { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 2px 5px
rgba(0,0,0,0.1); }
button { background: #007bff; color: white; border: none; padding: 8px 12px; border-radius: 4px;
cursor: pointer; }
button:hover { background: #0056b3; }
tr:hover { background: #f0f0f0; }

@media (max-width: 600px) {
  .grid-container { grid-template-columns: 1fr; }
}
```

3.script.js

```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
document.getElementById("taskForm").addEventListener("submit", async function(e) {
  e.preventDefault();
  let title = document.getElementById("title").value;
  let date = document.getElementById("date").value;
  let priority = document.getElementById("priority").value;
  let completed = document.getElementById("completed").checked;
  let hours = document.getElementById("hours").value;

  navigator.geolocation.getCurrentPosition(async (pos) => {
    let location = `${pos.coords.latitude.toFixed(2)}, ${pos.coords.longitude.toFixed(2)}`;

    let newTask = {
      id: Date.now(),
      title,
      date,
      priority,
      location,
      completed,
      hours
    };
    try {
      await saveTaskAsync(newTask);
      tasks.push(newTask);
      localStorage.setItem("tasks", JSON.stringify(tasks));
      updateTable();
      updateStats();
    } catch (err) {
      alert("Error saving task: " + err);
    }
  }, (err) => {
    alert("Location error: " + err.message);
```

```
  });
});
function saveTaskAsync(task) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (Math.random() > 0.1) resolve("Saved");
      else reject("Server error");
    }, 500);
  });
}

function updateTable() {
  let table = document.getElementById("taskTable");
  table.innerHTML =
"<tr><th>ID</th><th>Title</th><th>Priority</th><th>Location</th><th>Status</th></tr>";
  tasks.map(t => {
    let row = table.insertRow();
    row.insertCell(0).innerText = t.id;
    row.insertCell(1).innerText = t.title;
    row.insertCell(2).innerText = t.priority;
    row.insertCell(3).innerText = t.location || "N/A";
    row.insertCell(4).innerText = t.completed ? "Done" : "Pending";

    // Priority styling
    if (t.priority === "High") row.style.background = "#ffe0e0";
    if (t.priority === "Medium") row.style.background = "#fffbe0";
    if (t.priority === "Low") row.style.background = "#e0ffe0";
  });
}

function updateStats() {
  let done = tasks.filter(t => t.completed).length;
  let progress = tasks.length ? (done / tasks.length) * 100 : 0;
  document.getElementById("taskProgress").value = progress;

  let highPriority = tasks.filter(t => t.priority === "High").length;
  let ratio = tasks.length ? highPriority / tasks.length : 0;
  document.getElementById("priorityMeter").value = ratio;
}

window.onload = function() {
  updateTable();
  updateStats();
};
```